

Chapter 6. Monitoring

A production-ready microservice is one that is properly monitored. Proper monitoring is one of the most important parts of building a production-ready microservice and guarantees higher microservice availability. In this chapter, the essential components of microservice monitoring are covered, including which key metrics to monitor, how to log key metrics, building dashboards that display key metrics, how to approach alerting, and on-call best practices.

Principles of Microservice Monitoring

The majority of outages in a microservice ecosystem are caused by bad deployments. The second most common cause of outages is the lack of proper *monitoring*. It's easy to see why this is the case. If the state of a microservice is unknown, if key metrics aren't tracked, then any precipitating failures will remain unknown until an actual outage occurs. By the time a microservice experiences an outage due to lack of monitoring, its availability has already been compromised. During these outages, the time to mitigation and time to repair are prolonged, pulling the availability of the microservice down even further: without easily accessible information about the microservice's key metrics, developers are often faced with a blank slate, unprepared to quickly resolve the issue. This is why proper monitoring is essential: it provides the development team with all of the relevant information about the microservice. When a microservice is properly monitored, its state is never unknown.

Monitoring a production-ready microservice has four components. The first is proper *logging* of all relevant and important information, which allows developers to understand the state of the microservice at any time in the present or in the past. The second is the use of well-designed *dashboards* that accurately reflect the health of the microservice, and are organized in such a way that anyone at the company could view the dashboard and understand the health and status of the microservice without difficulty. The third component is actionable and effective *alerting* on all key metrics, a practice that makes it easy for developers to mitigate and

resolve problems with the microservice before they cause outages. The final component is the implementation and practice of running a sustainable *on-call rotation* responsible for the monitoring of the microservice. With effective logging, dashboards, alerting, and on-call rotation, the microservice's availability can be protected: failures and errors will be detected, mitigated, and resolved before they bring down any part of the microservice ecosystem.

A PRODUCTION-READY SERVICE IS PROPERLY MONITORED

- Its key metrics are identified and monitored at the host, infrastructure, and microservice levels.
- It has appropriate logging that accurately reflects the past states of the microservice.
- Its dashboards are easy to interpret, and contain all key metrics.
- Its alerts are actionable and are defined by signal-providing thresholds.
- There is a dedicated on-call rotation responsible for monitoring and responding to any incidents and outages.
- There is a clear, well-defined, and standardized on-call procedure in place for handling incidents and outages.

Key Metrics

Before we jump into the components of proper monitoring, it's important to identify precisely *what* we want and need to monitor: we want to monitor a microservice, but what does that *actually* mean? A microservice isn't an individual object that we can follow or track, it cannot be isolated and quarantined—it's far more complicated than that. Deployed across dozens, if not hundreds, of servers, the behavior of a microservice is the sum of its behavior across all of its instantiations, which isn't the easiest thing to quantify. The key is identifying which properties of a microservice are necessary and sufficient for describing its behavior, and then determining what changes in those properties tell us about the overall status and health of the microservice. We'll call these properties *key metrics*.

There are two types of key metrics: host and infrastructure metrics, and microservice metrics. Host and infrastructure metrics are those that pertain to the status of the infrastructure and the servers on which the microservice is running, while microservice metrics are metrics that are unique to the individual microservice. In terms of the four-layer model of the microservice ecosystem as described in [Chapter 1, *Microservices*](#), host and infrastructure metrics are metrics belonging to layers 1–3, while microservice metrics are those belonging to layer 4.

Separating key metrics into these two different types is important both organizationally and technically. Host and infrastructure metrics often affect more than one microservice: for example, if there is a problem with a particular server, and the microservice ecosystem shares the hardware resources among multiple microservices, host-level key metrics will be relevant to every microservice team that has a microservice deployed to that host. Likewise, microservice-specific metrics will rarely be applicable or useful to anyone but the team of developers working on that particular microservice. Teams should monitor both types of key metrics (that is, all metrics relevant to their microservice), and any metrics relevant to multiple microservices should be monitored and shared between the appropriate teams.

The host and infrastructure metrics that should be monitored for each microservice are the CPU utilized by the microservice on each host, the RAM utilized by the microservice on each host, the available threads, the microservice's open file descriptors (FD), and the number of database connections that the microservice has to any databases it uses. Monitoring these key metrics should be done in such a way that the status of each metric is accompanied by information about the infrastructure and the microservice. This means that monitoring should be granular enough that developers can know the status of the keys metrics for their microservice on any particular host and across all of the hosts that it runs on. For example, developers should be able to know how much CPU their microservice is using on one particular host *and* how much CPU their microservice is using across all hosts it runs on.

Some microservice ecosystems may use cluster management applications (like Mesos) in which the resources (CPU, RAM, etc.) are abstracted away from the host level. Host-level metrics won't be available in the same way to developers in these situations, but all key metrics for the microservice overall should still be monitored by the microservice team.

Determining the necessary and sufficient key metrics at the microservice level is a bit more complicated because it can depend on the particular language that the microservice is written in. Each language comes with its own special way of processing tasks, for example, and these language-specific features must be monitored closely in the majority of cases. Consider a Python service that utilizes uwsgi workers: the number of uwsgi workers is a necessary key metric for proper monitoring.

In addition to language-specific key metrics, we also must monitor the availability of the service, the service-level agreement (SLA) of the service, latency (of both the service as a whole and its API endpoints), success of API endpoints, responses and average response times of API endpoints, the services (clients) from which API requests originate (along with which endpoints they send requests to), errors and exceptions (both handled and unhandled), and the health and status of dependencies.

Importantly, all key metrics should be monitored everywhere that the application is deployed. This means that every stage of the deployment pipeline should be monitored. Staging must be closely monitored in order to catch any problems before a new candidate for production (a new build) is deployed to servers running production traffic. It almost goes without saying that all deployments to production servers should be monitored carefully, both in the canary and production deployment phases. (For more information on deployment pipelines, see [Chapter 3, Stability and Reliability](#).)

Once the key metrics for a microservice have been identified, the next step is to capture the metrics emitted by your service. Capture them, and then log them, graph them, and alert on them. We'll cover each of these steps in the following sections.

Host and infrastructure key metrics:

- CPU
- RAM
- Threads
- File descriptors
- Database connections

Microservice key metrics:

- Language-specific metrics
 - Availability
 - SLA
 - Latency
 - Endpoint success
 - Endpoint responses
 - Endpoint response times
 - Clients
 - Errors and exceptions
 - Dependencies
-

Logging

Logging is the first component of production-ready monitoring. It begins and belongs in the codebase of each microservice, nestled deep within the code of each service, capturing all of the information necessary to describe the state of the microservice. In fact, describing the state of the microservice at any given time in the recent past is the ultimate goal of logging.

One of the benefits of microservice architecture is the freedom it gives developers to deploy new features and code changes frequently, and one of the consequences of this newfound developer freedom and increased development velocity is that the microservice is always changing. In most cases, the service will not be the same service it was 12 hours ago, let alone several days ago, and reproducing any problems will be impossible. When faced with a problem, often the only way to determine the root

cause of an incident or outage is to comb through the logs, discover the state of the microservice at the time of the outage, and figure out why the service failed in that state. Logging needs to be such that developers can determine from the logs exactly what went wrong and where things fell apart.

LOGGING WITHOUT MICROSERVICE VERSIONING

Microservice versioning is often discouraged because it can lead to other (client) services pinning to specific versions of a microservice that may not be the best or most updated version of the microservice. Without versioning, determining the state of a microservice when a failure or outage occurred can be difficult, but thorough logging can prevent this from becoming a problem: if the logging is good enough that state of a microservice at the *time* of an outage can be sufficiently known and understood, the lack of versioning ceases to be a hindrance to quick and effective mitigation and resolution.

Determining precisely *what* to log is specific to each microservice. The best guidance on determining what needs to be logged is, somewhat unfortunately, necessarily vague: log whatever information is essential to describing the state of the service at a given time. Luckily, we can narrow down which information is necessary by restricting our logging to whatever can be contained in the code of the service. Host-level and infrastructure-level information won't (and shouldn't) be logged by the application itself, but by services and tools running the application platform. Some microservice-level key metrics and information, like hashed user IDs and request and response details can and should be located in the microservice's logs.

There are, of course, some things that *should never, ever be logged*. Logs should never contain identifying information, such as names of customers, Social Security numbers, and other private data. They should never contain information that could present a security risk, such as passwords, access keys, or secrets. In most cases, even seemingly innocuous things like user IDs and usernames should not be logged unless encrypted.

At times, logging at the individual microservice level will not be enough. As we've seen throughout this book, microservices do not live alone, but

within complex chains of clients and dependencies within the microservice ecosystem. While developers can try their best to log and monitor everything important and relevant to their service, tracking and logging requests and responses throughout the entire client and dependency chains from end-to-end can illuminate important information about the system that would otherwise go unknown (such as total latency and availability of the stack). To make this information accessible and visible, building a production-ready microservice ecosystem requires tracing each request through the entire stack.

The reader might have noticed at this point that it appears that a lot of information needs to be logged. Logs are data, and logging is expensive: they are expensive to store, they are expensive to access, and both storing and accessing logs comes with the additional cost associated with making expensive calls over the network. The cost of storing logs may not seem like much for an individual microservice, but if the logging needs of all the microservices within a microservice ecosystem are added together, the cost is rather high.

LOGS AND DEBUGGING

Avoid adding debugging logs in code that will be deployed to production—such logs are very costly. If any logs are added specifically for the purpose of debugging, developers should take great care to ensure that any branch or build containing these additional logs does not ever touch production.

Logging needs to be scalable, it needs to be available, and it needs to be easily accessible *and* searchable. To keep the cost of logs down and to ensure scalability and high availability, it's often necessary to impose per-service logging quotas along with limits and standards on what information can be logged, how many logs each microservice can store, and how long the logs will be stored before being deleted.

Dashboards

Every microservice must have at least one *dashboard* where all key metrics (such as hardware utilization, database connections, availability, latency, responses, and the status of API endpoints) are collected and dis-

played. A dashboard is a graphical display that is updated in real time to reflect all the most important information about a microservice. Dashboards should be easily accessible, centralized, and standardized across the microservice ecosystem.

Dashboards should be easy to interpret so that an outsider can quickly determine the health of the microservice: anyone should be able to look at the dashboard and know immediately whether or not the microservice is working correctly. This requires striking a balance between overloading a viewer with information (which would render the dashboard effectively useless) and not displaying enough information (which would also make the dashboard useless): only the necessary minimum of information about key metrics should be displayed.

A dashboard should also serve as an accurate reflection of the overall quality of monitoring of the entire microservice. Any key metric that is alerted on should be included in the dashboard (we will cover this in the next section): the exclusion of any key metric in the dashboard will reflect poor monitoring of the service, while the inclusion of metrics that are not necessary will reflect a neglect of alerting (and, consequently, monitoring) best practices.

There are several exceptions to the rule against inclusion of nonkey metrics. In addition to key metrics, information about each phase of the deployment pipeline should be displayed, though not necessarily within the same dashboard. Developers working on microservices that require monitoring a large number of key metrics may opt to set up separate dashboards for each deployment phase (one for staging, one for canary, and one for production) to accurately reflect the health of the microservice at each deployment phase: since different builds will be running on the deployment phases simultaneously, accurately reflecting the health of the microservice in a dashboard might require approaching dashboard design with the goal of reflecting the health of the microservice at a particular deployment phase (treating them almost as different microservices, or at least as different instantiations of a microservice).

Even though dashboards can illuminate anomalies and negative trends of a microservice's key metrics, developers should never need to watch a microservice's dashboard in order to detect incidents and outages. Doing so is an anti-pattern that leads to deficiencies in alerting and overall monitoring.

To assist in determining problems introduced by new deployments, it helps to include information about when a deployment occurred in the dashboard. The most effective and useful way to accomplish this is to make sure that deployment times are shown within the graphs of each key metric. Doing so allows developers to quickly check graphs after each deployment to see if any strange patterns emerge in any of the key metrics.

Well-designed dashboards also give developers an easy, visual way to detect anomalies and determine alerting thresholds. Very slight or gradual changes or disturbances in key metrics run the risk of not being caught by alerting, but a careful look at an accurate dashboard can illuminate anomalies that would otherwise go undetected. Alerting thresholds, which we will cover in the next section, are notoriously difficult to determine, but can be set appropriately when historical data on the dashboard is examined: developers can see normal patterns in key metrics, view spikes in metrics that occurred with outages (or led to outages) in the past, and then set thresholds accordingly.

Alerting

The third component of monitoring a production-ready microservice is real-time *alerting*. The detection of failures, as well as the detection of changes within key metrics that could lead to a failure, is accomplished through alerting. To ensure this, all key metrics—host-level metrics, infrastructure metrics, and microservice-specific metrics—should be alerted on, with alerts set at various thresholds. Effective and actionable alerting is essential to preserving the availability of a microservice and preventing downtime.

Setting up Effective Alerting

Alerts must be set up for all key metrics. Any change in a key metric at the host level, infrastructure level, or microservice level that could lead to an outage, cause a spike in latency, or somehow harm the availability of the microservice should trigger an alert. Importantly, alerts should also be triggered whenever a key metric is *not* seen.

All alerts should be useful: they should be defined by good, signal-providing thresholds. Three types of thresholds should be set for each key metric, and have both upper and lower bounds: *normal*, *warning*, and *critical*. Normal thresholds reflect the usual, appropriate upper and lower bounds of each key metric and shouldn't ever trigger an alert. Warning thresholds on each key metric will trigger alerts when there is a deviation from the norm that could lead to a problem with the microservice; warning thresholds should be set such that they will trigger alerts *before* any deviations from the norm cause an outage or otherwise negatively affect the microservice. Critical thresholds should be set based on which upper and lower bounds on key metrics actually cause an outage, cause latency to spike, or otherwise hurt a microservice's availability. In an ideal world, warning thresholds should trigger alerts that lead to quick detection, mitigation, and resolution before any critical thresholds are reached. In each category, thresholds should be high enough to avoid noise, but low enough to catch any and all real problems with key metrics.

DETERMINING THRESHOLDS EARLY IN THE LIFECYCLE OF A MICROSERVICE

Thresholds for key metrics can be very difficult to set without historical data. Any thresholds set early in a microservice's lifecycle run the risk of either being useless or triggering too many alerts. To determine the appropriate thresholds for a new microservice (or even an old one), developers can run load testing on the microservice to gauge where the thresholds should lie. Running "normal" traffic loads through the microservice can determine the normal thresholds, while running larger-than-expected traffic loads can help determine warning and critical thresholds.

All alerts need to be actionable. Nonactionable alerts are those that are triggered and then resolved (or ignored) by the developer(s) on call for the microservice because they are not important, not relevant, do not signify that anything is wrong with the microservice, or alert on a problem that cannot be resolved by the developer(s). Any alert that cannot be im-

mediately acted on by the on-call developer(s) should be removed from the pool of alerts, reassigned to the relevant on-call rotation, or (if possible) changed so that it becomes actionable.

Some of the key microservice metrics run the risk of being nonactionable. For example, alerting on the availability of dependencies can easily lead to nonactionable alerts if dependency outages, increases in dependency latency, or dependency downtime do not require any action to be taken by their client(s). If no action needs to be taken, then the thresholds should be set appropriately, or in more extreme cases, no alerts should be set on dependencies at all. However, if any action at all should be taken, even something as small as contacting the dependency's on-call or development team in order to alert them to the issue and/or coordinate mitigation and resolution, then an alert should be triggered.

Handling Alerts

Once an alert has been triggered, it needs to be handled quickly and effectively. The root cause of the triggered alert should be mitigated and resolved. To quickly and effectively handle alerts, there are several steps that can be taken.

The first step is to create step-by-step instructions for each known alert that detail how to triage, mitigate, and resolve each alert. These step-by-step alert instructions should live within an on-call runbook within the centralized documentation of each microservice, making them easily accessible to anyone who is on call for the microservice (more details on runbooks can be found in [Chapter 7, *Documentation and Understanding*](#)). Runbooks are crucial to the monitoring of a microservice: they allow any on-call developer to have step-by-step instructions on how to mitigate and resolve the root causes of each alert. Since each alert is tied to a deviation in a key metric, runbooks can be written so that they address each key metric, known causes of deviations from the norm, and how to go about debugging the problem.

Two types of on-call runbooks should be created. The first are runbooks for host-level and infrastructure-level alerts that should be shared between the whole engineering organization—these should be written for every key host-level and infrastructure-level metric. The second are on-

call runbooks for specific microservices that have step-by-step instructions regarding microservice-specific alerts triggered by changes in key metrics; for example, a spike in latency should trigger an alert, and there should be step-by-step instructions in the on-call runbook that clearly document how to debug, mitigate, and resolve spikes in the microservice's latency.

The second step is to identify alerting anti-patterns. If the microservice on-call rotation is overwhelmed by alerts yet the microservice appears to work as expected, then any alerts that are seen more than once but that can be easily mitigated and/or resolved should be automated away. That is, build the mitigation and/or resolution steps into the microservice itself. This holds for every alert, and writing step-by-step instructions for alerts within on-call runbooks allows executing on this strategy to be rather effective. In fact, any alert that, once triggered, requires a simple set of steps to be taken in order to be mitigated and resolved, can be easily automated away. Once this level of production-ready monitoring has been established, a microservice should never experience the same exact problem twice.

On-Call Rotations

In a microservice ecosystem, the development teams themselves are responsible for the availability of their microservices. Where monitoring is concerned, this means that developers need to be on call for their own microservices. The goal of each developer on-call for a microservice needs to be clear: they are to detect, mitigate, and resolve any issue that arises with the microservice during their on call shift before the issue causes an outage for their microservice or impacts the business itself.

In some larger engineering organizations, site reliability engineers, DevOps, or other operations engineers may take on the responsibility for monitoring and on call, but this requires each microservice to be relatively stable and reliable before the on-call responsibilities can be handed off to another team. In most microservice ecosystems, microservices rarely reach this high level of stability because, as we've seen throughout the previous chapters, microservices are constantly changing. In a microservice ecosystem, developers need to bear the responsibility of monitoring the code that they deploy.

Designing good on-call rotations is crucial and requires the involvement of the entire team. To prevent burnout, on-call rotations should be both brief and shared: no fewer than two developers should ever be on call at one time, and on-call shifts should last no longer than one week and be spaced no more frequently than one month apart.

The on-call rotations of each microservice should be internally publicized and easily accessible. If a microservice team is experiencing issues with one of their dependencies, they should be able to track down the on-call engineers for the microservice and contact them very quickly. Hosting this information in a centralized place helps to make developers more effective in triaging problems and preventing outages.

Developing standardized on-call procedures across an engineering organization will go a long way toward building a sustainable microservice ecosystem. Developers should be trained about how to approach their on-call shifts, be made aware of on-call best practices, and be ramped up for joining the on-call rotation very quickly. Standardizing this process and making on-call expectations completely clear to every developer will prevent the burnout, confusion, and frustration that usually accompanies any mention of joining an on-call rotation.

Evaluate Your Microservice

Now that you have a better understanding of monitoring, use the following list of questions to assess the production-readiness of your microservice(s) and microservice ecosystem. The questions are organized by topic, and correspond to the sections within this chapter.

Key Metrics

- What are this microservice's key metrics?
- What are the host and infrastructure metrics?
- What are the microservice-level metrics?
- Are all the microservice's key metrics monitored?

Logging

- What information does this microservice need to log?

- Does this microservice log all important requests?
- Does the logging accurately reflect the state of the microservice at any given time?
- Is this logging solution cost-effective and scalable?

Dashboards

- Does this microservice have a dashboard?
- Is the dashboard easy to interpret? Are all key metrics displayed on the dashboard?
- Can I determine whether or not this microservice is working correctly by looking at the dashboard?

Alerting

- Is there an alert for every key metric?
- Are all alerts defined by good, signal-providing thresholds?
- Are alert thresholds set appropriately so that alerts will fire before an outage occurs?
- Are all alerts actionable?
- Are there step-by-step triage, mitigation, and resolution instructions for each alert in the on-call runbook?

On-Call Rotations

- Is there a dedicated on-call rotation responsible for monitoring this microservice?
- Is there a minimum of two developers on each on-call shift?
- Are there standardized on-call procedures across the engineering organization?

